

ARRAY QNO:2

QUESTION:

Problem Description: Ram has provide inputs two numbers 'p' and 'q' to Sakthi. He wants to creates a matrix of size p x q (p rows and q columns) in which every elements is either Y or O. The Ys and Os must be filled alternatively, the matrix should have outermost rectangle of Ys, then a rectangle of Os, then a rectangle of Ys, and so on..

Constraints:

1 <= p, q <= 1000

Input Format:

Input lines must be how many rows and columns in that matrix, also values must be separate space.

Output Format:

Print the output in a separate lines.

Logical Test Cases

Test Case 1

INPUT (STDIN)

6 7

Test Case 2

INPUT (STDIN)

5 8

EXPECTED OUTPUT

```
Y Y Y Y Y Y Y
Y 0 0 0 0 0 Y
Y 0 Y Y Y 0 Y
Y 0 Y Y Y 0 Y
Y 0 0 0 0 0 Y
Y Y Y Y Y Y Y
```

EXPECTED OUTPUT

```
Y Y Y Y Y Y Y Y
Y 0 0 0 0 0 0 Y
Y 0 Y Y Y Y 0 Y
Y 0 0 0 0 0 0 Y
Y Y Y Y Y Y Y Y
```

▼ Mandatory Test Cases

Test Case 1
KEYWORD
<pre>while(top<=bottom && right>=left)</pre>

▼ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
12	290	52

Source Code:

```
#include <stdio.h>

char matrix[1005][1005];

int main() {
    int p, q;

    if (scanf("%d %d", &p, &q) != 2) {
        return 0;
    }

    int top = 0;
    int bottom = p - 1;
    int left = 0;
    int right = q - 1;
    char ch = 'Y';

    while(top<=bottom && right>=left) {
        for (int i = left; i <= right; i++) {
            matrix[top][i] = ch;
        }
        top++;

        for (int i = top; i <= bottom; i++) {
            matrix[i][right] = ch;
        }
        right--;
    }
}
```

```

    if (top <= bottom) {
        for (int i = right; i >= left; i--) {
            matrix[bottom][i] = ch;
        }
        bottom--;
    }

    if (left <= right) {
        for (int i = bottom; i >= top; i--) {
            matrix[i][left] = ch;
        }
        left++;
    }

    if (ch == 'Y') {
        ch = '0';
    } else {
        ch = 'Y';
    }
}

for (int i = 0; i < p; i++) {
    for (int j = 0; j < q; j++) {
        printf("%c ", matrix[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Output:

```

6 7
Y Y Y Y Y Y Y
Y 0 0 0 0 0 Y
Y 0 Y Y Y 0 Y
Y 0 Y Y Y 0 Y
Y 0 0 0 0 0 Y
Y Y Y Y Y Y Y

```

```

5 8
Y Y Y Y Y Y Y Y
Y 0 0 0 0 0 0 Y
Y 0 Y Y Y Y 0 Y
Y 0 0 0 0 0 0 Y
Y Y Y Y Y Y Y Y

```